

# Guia general para la redaccion de una tesis en Ciencias de la Computacion

Plantilla conceptual — no ligada a una nomenclatura fija de capitulos

## Resumen

Este documento no es una tesis: es un *molde argumentativo*, un ensayo que explica que debe **existir** dentro de una tesis de Ciencias de la Computacion para que esta sea correcta, autocontenida y defendible, con independencia de los nombres que finalmente reciban sus capitulos. La tesis se entiende aqui como lo describen Booth et al. (2016): una respuesta justificada a una pregunta que le importa a una comunidad. Sobre esa base se identifican cuatro funciones que todo trabajo de investigacion en computacion debe satisfacer — situar el problema, sostenerse a si mismo con la teoria necesaria, distinguirse de lo ya hecho, y ofrecer una contribucion con evidencia de que funciona — y se cierra con la sintesis final. El objetivo no es imponer una estructura rigida sino asegurar que, sin importar como se titulen las secciones, la *esencia* de cada funcion este presente.

## Índice

|                                                                                        |          |
|----------------------------------------------------------------------------------------|----------|
| <b>1. Como leer y usar este molde</b>                                                  | <b>2</b> |
| <b>2. Bloque (a): Introduccion — situar el problema</b>                                | <b>2</b> |
| 2.1. Contextualizacion y planteamiento del problema . . . . .                          | 3        |
| 2.2. Objetivos, preguntas de investigacion e hipotesis . . . . .                       | 3        |
| 2.3. Alcance, supuestos y aporte esperado . . . . .                                    | 3        |
| <b>3. Bloque (b): Fundamentos necesarios y justificacion frente al estado del arte</b> | <b>4</b> |
| 3.1. Fundamentos teoricos (autocontencion) . . . . .                                   | 4        |
| 3.2. Estado del arte y justificacion frente a lo existente . . . . .                   | 4        |
| 3.3. La brecha como bisagra . . . . .                                                  | 5        |
| <b>4. Bloque (c): Presentacion de la contribucion y evidencia</b>                      | <b>5</b> |
| 4.1. Presentacion formal de la contribucion . . . . .                                  | 5        |
| 4.2. Diseño experimental (capitulo condicional) . . . . .                              | 6        |
| 4.3. Resultados e interpretacion . . . . .                                             | 7        |
| <b>5. Bloque (d): Conclusion</b>                                                       | <b>7</b> |
| 5.1. Sintesis de la contribucion . . . . .                                             | 7        |
| 5.2. Limitaciones . . . . .                                                            | 7        |
| 5.3. Trabajo futuro e implicancias . . . . .                                           | 8        |

# 1 Como leer y usar este molde

Una tesis de computacion no se organiza por convencion sino porque cada parte cumple un rol argumentativo especifico. Zobel (2014) insiste en que escribir en computacion es, ante todo, *construir un argumento verificable*: cada afirmacion debe apoyarse en evidencia, ya sea una demostracion, un experimento o una referencia. Este documento traduce esa idea en cuatro bloques funcionales que pueden llamarse de muchas maneras (“Capitulo 1”, “Introduccion”, “Marco teorico”, “Estado del arte”, “Metodologia”, “Resultados”, etc.) pero cuya **funcion** no es negociable.

## Nota metodologica

Este documento esta escrito como ensayo argumentativo, no como tabla de contenidos vacia. Cada bloque se explica indicando (i) que problema resuelve dentro de la arquitectura global de la tesis, (ii) que elementos minimos debe contener, (iii) por que la literatura metodologica exige esos elementos, y (iv) cuando un elemento es opcional u obligatorio segun el tipo de contribucion (algoritmo, modelo formal, sistema, estudio empirico).

La fig. 1 resume el flujo. Notese que las flechas no son solo de avance: una tesis madura revisa constantemente sus primeros bloques a la luz de lo que descubre en los ultimos, un proceso iterativo de refinamiento que Booth et al. (2016) describen como caracteristico de toda investigacion seria, y que en la figura se representa mediante la flecha punteada de retroalimentacion.

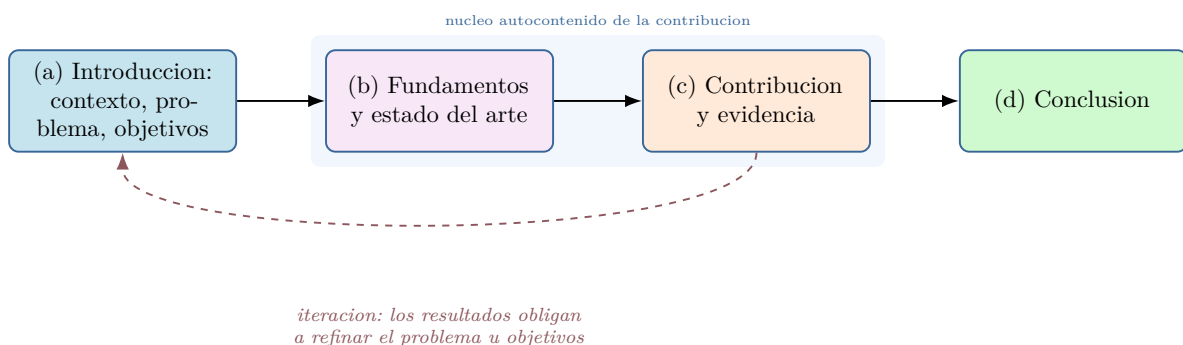


Figura 1: Flujo funcional de una tesis de Ciencias de la Computacion. Los nombres de los capitulos pueden variar; las funciones (a)–(d) no.

## 2 Bloque (a): Introduccion — situar el problema

La primera funcion de la tesis es **situar al lector**: por que este problema importa, que se sabe y que no, y que se propone hacer al respecto. J. W. Creswell y J. D. Creswell (2018)

formaliza este bloque como la articulacion de cuatro componentes que rara vez faltan en un proyecto de investigacion serio: el planteamiento del problema, la revision preliminar de literatura que motiva ese planteamiento, el proposito del estudio y las preguntas o hipotesis que lo operacionalizan. Ninguno de estos componentes exige un nombre de seccion especifico, pero los cuatro deben poder señalarse con el dedo en algun lugar del documento.

## 2.1 Contextualizacion y planteamiento del problema

Debe explicarse el dominio de aplicacion o el area teorica, y dentro de ese dominio, delimitar con precision el problema. Booth et al. (2016, cap. 2) recomienda formular el problema como una tension no resuelta: algo que *deberia* poder hacerse (o entenderse, o demostrarse) y que actualmente no se puede, no se sabe hacer bien, o no se sabe hacer de forma eficiente, correcta o segura. Esta tension es lo que en la jerga de investigacion se llama la **brecha** (*gap*); situarla aqui evita que aparezca recien en el capitulo de estado del arte, donde ya seria tardio.

## 2.2 Objetivos, preguntas de investigacion e hipotesis

Todo objetivo debe ser verificable por el propio diseño de la contribucion que se presentara mas adelante: si el objetivo es “diseñar un algoritmo mas eficiente”, el bloque de evidencia debera demostrar esa eficiencia; si es “demostrar una propiedad formal”, debera existir una prueba. Segun el tipo de investigacion, esta seccion puede articularse como:

- **Objetivo general y objetivos especificos**, tipico en trabajos de diseño o construccion de sistemas y algoritmos.
- **Preguntas de investigacion** (*research questions*), frecuentes en estudios empiricos de ingenieria de software, siguiendo el patron de formulacion que describen Wohlin et al. (2012).
- **Hipotesis formales**, cuando el trabajo es de naturaleza cuantitativa o experimental y se someteran a prueba estadistica.

### Error frecuente

Un objetivo que no puede fallar no es un objetivo cientifico. Si no existe ninguna evidencia posible que pudiera mostrar que la contribucion *no* funciona, el objetivo esta mal planteado. Esta es una de las advertencias mas repetidas por Zobel (2014): la investigacion en computacion debe ser **falsable o refutable en la practica**, no solo plausible en el discurso.

## 2.3 Alcance, supuestos y aporte esperado

Cerrando el bloque, conviene anticipar brevemente en que consiste la contribucion (sin desarrollarla aun) y cuales son sus limites deliberados: que se asume, que queda fuera del alcance y por que. Esto no sustituye a la conclusion; es una promesa que el resto de la tesis debe cumplir.

### 3 Bloque (b): Fundamentos necesarios y justificación frente al estado del arte

Esta es la parte que garantiza que la tesis sea **autocontenida**: un lector competente en computación, pero no necesariamente experto en el subtema, debe poder entender la contribución sin salir del documento. Al mismo tiempo, debe quedar claro por qué la contribución es necesaria frente a lo que ya existe. Aunque en muchos programas estas dos tareas se separan en capítulos distintos (“Marco teórico” y “Estado del arte”), conceptualmente responden a una misma pregunta: *¿qué necesito saber para entender por qué mi contribución es nueva y válida?*

#### 3.1 Fundamentos teóricos (autocontención)

Aquí se introduce el aparato conceptual mínimo e imprescindible: definiciones formales, notación, modelos computacionales, estructuras de datos, resultados previos que se usarán como caja negra, o el marco estadístico necesario para interpretar los experimentos. Denning et al. (1989) argumentan que la disciplina de la computación se sostiene sobre tres paradigmas — teoría, abstracción (modelado) y diseño — y que toda contribución puede describirse en términos de al menos uno de ellos; esta sección debe dar al lector las herramientas del paradigma que corresponda a la tesis.

##### Definición: nivel de autocontención exigible

Una tesis es autocontenida cuando, tras leer este bloque, el lector puede verificar por sí mismo cada paso técnico del bloque de contribución sin recurrir a fuentes externas para los conceptos *centrales*. Puede remitirse a fuentes externas para resultados *auxiliares* bien conocidos (por ejemplo, un teorema clásico de complejidad), siempre citando su origen.

Según la naturaleza de la contribución, el contenido mínimo cambia:

- **Contribución algorítmica**: modelo de cómputo, definiciones de complejidad temporal/espacial, estructuras de datos empleadas.
- **Contribución formal (teorema, sistema lógico, semántica)**: definiciones, axiomas, lemas previos y convenciones notacionales completas, siguiendo las prácticas de claridad que recomienda Lamport (1995) para que cada paso de una demostración sea verificable.
- **Contribución de modelado** (arquitecturas, modelos probabilísticos, modelos de simulación): supuestos del modelo, variables, y criterios de validez.
- **Contribución de sistema/herramienta**: arquitectura general, decisiones de diseño y las tecnologías sobre las que se construye.

#### 3.2 Estado del arte y justificación frente a lo existente

No basta con listar trabajos relacionados: hay que **compararlos críticamente** contra los objetivos planteados en el Bloque (a) para exponer la brecha que la tesis llena. Kitchenham

y Charters (2007) formalizan este proceso mediante revisiones sistematicas de literatura, con criterios explicitos de busqueda, inclusion y exclusion; aun cuando una tesis no requiera una revision sistematica completa, adoptar su disciplina — fuentes trazables, criterios explicitos, cobertura representativa del area — da credibilidad a la afirmacion de novedad. Kuhn (1962, cap. 2) recuerda ademas que el valor de una contribucion cientifica se mide en relacion con el paradigma vigente: una tesis no ataca al vacio, ataca las limitaciones concretas de soluciones concretas.

#### Buena practica

Organice la comparacion en una tabla o taxonomia (por ejemplo, por enfoque, supuestos, complejidad, o resultados reportados) y ubique explicitamente su propia contribucion como una fila o celda adicional de esa misma tabla. Esto hace visible la brecha en lugar de solo describirla en prosa.

#### Error frecuente

Evite el “estado del arte panoramico”: enumerar decenas de trabajos sin conectarlos con los objetivos de la tesis. Cada trabajo citado en esta seccion debe responder a una de dos preguntas: ¿por que no resuelve el problema planteado en el Bloque (a)? o ¿que herramienta metodologica le presta a esta tesis?

### 3.3 La brecha como bisagra

La brecha (*gap*) identificada aqui debe coincidir exactamente con la tension descrita en la Seccion 2.1. Si no coinciden, alguno de los dos bloques esta mal planteado. Esta coherencia es, en la practica, el criterio mas comun de rechazo en comites de tesis: objetivos que prometen algo que el estado del arte revisado no muestra como necesario, o un estado del arte que revela una brecha distinta de la que los objetivos atacan.

## 4 Bloque (c): Presentacion de la contribucion y evidencia

Este es el nucleo original de la tesis: aqui se presenta *que se hizo* y se demuestra *que funciona*. La forma que tome depende enteramente del tipo de contribucion, pero la exigencia de fondo es constante: toda afirmacion de merito debe venir acompañada de evidencia de la clase adecuada a esa afirmacion (Zobel, 2014).

### 4.1 Presentacion formal de la contribucion

- **Algoritmo:** especificacion precisa (pseudocodigo o formalismo equivalente), argumento de correctitud, y analisis de complejidad. Un algoritmo sin analisis de complejidad ni prueba de correctitud es una receta, no una contribucion cientifica.
- **Teorema o resultado formal:** enunciado preciso, demostracion completa y explicita. Lamport (1995) propone estructurar toda demostracion como una jerarquia de pasos

numerados, cada uno lo bastante pequeño para ser verificado de forma mecánica; esta estructura facilita enormemente la revisión por el comité.

- **Modelo o arquitectura:** descripción formal del modelo, justificación de cada supuesto, y — cuando corresponda — un argumento de por qué el modelo captura los aspectos relevantes del fenómeno.
- **Sistema o artefacto de software:** arquitectura, decisiones de diseño relevantes, y una descripción suficiente para que el sistema sea reproducible por terceros.

## 4.2 Diseño experimental (capítulo condicional)

### Nota metodológica: condicionalidad de la evaluación experimental

Si la contribución hace afirmaciones de *desempeño*, *eficiencia práctica*, *eficacia*, o cualquier propiedad no demostrable de forma puramente analítica, entonces un capítulo o sección de experimentos es **obligatorio**. Si la contribución es puramente formal y sus propiedades quedan completamente establecidas por demostración (por ejemplo, un algoritmo cuya complejidad se deriva analíticamente y cuya corrección se prueba matemáticamente), la evaluación experimental es opcional y puede limitarse a validar empíricamente el análisis teórico.

Cuando el capítulo experimental es necesario, su diseño no puede ser improvisado. Wohlin et al. (2012) y, antes, Basili et al. (1986, metodología seminal en ingeniería de software), establecen que un experimento defendible requiere, como mínimo:

1. **Definición del objetivo experimental**, ligado directamente a las preguntas de investigación del Bloque (a).
2. **Variables independientes y dependientes**, y las métricas concretas con que se miden.
3. **Diseño experimental** (grupos de control, líneas base de comparación, aleatorización cuando aplique).
4. **Amenazas a la validez** (interna, externa, de constructo y de conclusión), reportadas de forma explícita y no solo como formalidad.
5. **Análisis estadístico** apropiado al tipo de dato recolectado.

Para experimentos de rendimiento de sistemas (tiempos de ejecución, uso de memoria, escalabilidad), Jain (1991) sigue siendo la referencia estándar sobre metodología de medición, caracterización de carga de trabajo y presentación honesta de resultados: en particular, advierte contra reportar un único número (por ejemplo, un promedio) sin exponer variabilidad, condiciones del entorno de medición ni supuestos de la carga usada.

### Error frecuente

Una lista de baselines mal elegida invalida el capítulo entero. La línea base debe ser el mejor competidor razonable disponible para el mismo problema, no una versión

deliberadamente debil. Comparar solo contra una implementacion *naïve* propia, cuando existen alternativas publicadas del estado del arte, es una de las objeciones mas comunes en defensas de tesis de corte experimental.

### 4.3 Resultados e interpretacion

Los resultados deben presentarse separados de su interpretacion: primero los datos (tablas, graficas, medidas de dispersion), luego el analisis de lo que significan en relacion con las hipotesis u objetivos. La table 1 resume, a modo de referencia cruzada, que evidencia corresponde a cada tipo de contribucion.

| Tipo de contribucion       | Evidencia minima obligatoria                        | Evidencia adicional recomendada                    |
|----------------------------|-----------------------------------------------------|----------------------------------------------------|
| Algoritmo                  | Prueba de correctitud + analisis de complejidad     | Evaluacion empirica de rendimiento vs. baselines   |
| Teorema / resultado formal | Demostracion completa                               | Ejemplos ilustrativos, casos limite                |
| Modelo / arquitectura      | Justificacion de supuestos + validacion cualitativa | Validacion cuantitativa contra datos reales        |
| Sistema / herramienta      | Descripcion reproducible de la arquitectura         | Estudio de caso, benchmark, evaluacion de usuarios |
| Estudio empirico           | Diseño experimental riguroso + analisis estadistico | Replicacion, analisis de amenazas a la validez     |

Tabla 1: Correspondencia entre tipo de contribucion y evidencia exigible.

## 5 Bloque (d): Conclusion

El cierre de la tesis no es un resumen mecanico: es el lugar donde se cierra el argumento abierto en la Introduccion. Phillips y Pugh (2010) señalan que los comites evaluadores prestan especial atencion a que la conclusion responda, de forma explicita, a cada objetivo o pregunta planteada al inicio; dejar un objetivo sin respuesta clara — incluso si la respuesta es negativa o parcial — es motivo frecuente de observaciones.

### 5.1 Sintesis de la contribucion

Se recapitula, en un parrafo, que se propuso, que se hizo y que se demostro, evitando reintroducir informacion nueva. Debe leerse como el cierre logico de la promesa hecha en la Seccion 2.3.

### 5.2 Limitaciones

Toda contribucion tiene fronteras. Declarar limitaciones no debilita la tesis: la fortalece, porque demuestra que el autor entiende exactamente el alcance de lo demostrado (Booth et al., 2016). Las limitaciones deben distinguirse de trabajos futuros: una limitacion es algo que el

trabajo *actual* no cubre por diseño o por restriccion; el trabajo futuro es una direccion que otros (o el mismo autor) podrian seguir despues.

### 5.3 Trabajo futuro e implicancias

Se identifican extensiones razonables, abiertas por los propios resultados o por las limitaciones reconocidas. Cuando corresponda, tambien se discuten implicancias practicas o teoricas mas amplias del hallazgo, conectando de vuelta con la relevancia planteada en la Introduccion — cerrando asi el ciclo argumentativo completo de la tesis.

## 6 Consideraciones transversales de forma

Ademas del contenido, la forma en que se escribe cada bloque afecta su credibilidad. Zobel (2014) y Knuth et al. (1989) coinciden en un conjunto reducido de principios que aplican a los cuatro bloques por igual:

- **Precision terminologica:** cada termino tecnico se define la primera vez que se usa y se usa de forma consistente en adelante.
- **Notacion unica:** un mismo simbolo no debe significar dos cosas distintas en capitulos distintos.
- **Verificabilidad:** cualquier afirmacion cuantitativa debe poder rastrearse hasta un dato, una demostracion o una cita.
- **Progresion logica:** cada seccion debe poder justificar, en una oracion, por que aparece despues de la anterior y antes de la siguiente.

#### Ejemplo: prueba de coherencia entre bloques

Un ejercicio util antes de entregar la tesis es escribir, en una sola oracion por bloque, la funcion que cumple: (a) “expongo que X es un problema no resuelto y me propongo lograr Y”; (b) “explico lo necesario para entender Y, y muestro que nadie lo ha logrado bajo las condiciones que me propongo”; (c) “presento Z y demuestro/mido que Z logra Y”; (d) “confirmo que Z logro Y, dentro de los limites L, y sugiero W como siguiente paso”. Si estas cuatro oraciones no encajan entre si, hay un problema estructural que ninguna cantidad de redaccion cuidadosa en el detalle podra ocultar ante un comite atento.

## 7 Plantilla esqueleto de referencia

La siguiente estructura es un ejemplo de como los cuatro bloques funcionales suelen materializarse en capitulos; los nombres son intercambiables y pueden fusionarse o dividirse segun el programa de posgrado o pregrado correspondiente.

### Buena practica: esqueleto orientativo

1. **Introduccion** — contexto, problema, objetivos/hipotesis, alcance. (*Bloque a*)
2. **Fundamentos teoricos** — definiciones, notacion, resultados previos usados como base. (*Bloque b, parte teorica*)
3. **Estado del arte** — trabajos relacionados y brecha identificada. (*Bloque b, parte comparativa*)
4. **Contribucion propuesta** — algoritmo, modelo, teorema o sistema, con su justificacion formal. (*Bloque c, parte formal*)
5. **Diseño experimental y resultados** — *solo si aplica segun la table 1*. (*Bloque c, parte empirica*)
6. **Conclusiones** — sintesis, limitaciones, trabajo futuro. (*Bloque d*)

Finalmente, conviene recordar la advertencia de Wing (2006) sobre el pensamiento computacional: una tesis en esta disciplina no solo resuelve un problema puntual, sino que aporta una forma reutilizable de pensar sobre una clase de problemas. Mantener esa aspiracion presente en los cuatro bloques — y no solo en la introduccion — es lo que distingue una tesis solida de un reporte tecnico extenso.

## Referencias

- Basili, V. R., R. W. Selby y D. H. Hutchens (1986). «Experimentation in Software Engineering». En: *IEEE Transactions on Software Engineering* SE-12.7, págs. 733-743. DOI: [10.1109/TSE.1986.6312975](https://doi.org/10.1109/TSE.1986.6312975).
- Booth, W. C. et al. (2016). *The Craft of Research*. 4th. Chicago: University of Chicago Press.
- Creswell, J. W. y J. D. Creswell (2018). *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. 5th. Thousand Oaks, CA: SAGE Publications.
- Denning, P. J. et al. (1989). «Computing as a Discipline». En: *Communications of the ACM* 32.1, págs. 9-23. DOI: [10.1145/63238.63239](https://doi.org/10.1145/63238.63239).
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. New York: Wiley.
- Kitchenham, B. y S. Charters (2007). *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Inf. téc. EBSE-2007-01. EBSE, Keele University y University of Durham.
- Knuth, D. E., T. Larrabee y P. M. Roberts (1989). *Mathematical Writing*. MAA Notes 14. Washington, DC: Mathematical Association of America.
- Kuhn, T. S. (1962). *The Structure of Scientific Revolutions*. Chicago: University of Chicago Press.
- Lamport, L. (1995). «How to Write a Proof». En: *The American Mathematical Monthly* 102.7, págs. 600-608. DOI: [10.1080/00029890.1995.12004630](https://doi.org/10.1080/00029890.1995.12004630).
- Phillips, E. M. y D. S. Pugh (2010). *How to Get a PhD: A Handbook for Students and Their Supervisors*. 5th. Maidenhead: Open University Press / McGraw-Hill.
- Wing, J. M. (2006). «Computational Thinking». En: *Communications of the ACM* 49.3, págs. 33-35. DOI: [10.1145/1118178.1118215](https://doi.org/10.1145/1118178.1118215).
- Wohlin, C. et al. (2012). *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer. DOI: [10.1007/978-3-642-29044-2](https://doi.org/10.1007/978-3-642-29044-2).
- Zobel, J. (2014). *Writing for Computer Science*. 3rd. London: Springer. DOI: [10.1007/978-1-4471-6639-9](https://doi.org/10.1007/978-1-4471-6639-9).